

Section 80 Editorial — Undirected Connectivity Decomposition

A. Bridge Distance Queries

How to find it

Delete bridges conceptually, contract the remaining components, and answer distance in the resulting bridge tree.

Correctness

A DFS tree edge is a bridge exactly when its child subtree has no back edge reaching the parent or above. Contracting all nonbridge edges preserves precisely the separations caused by bridges. The quotient is a tree: a quotient cycle would give every edge on it an alternate route. Hence every route between two components crosses exactly the bridge-tree path, whose length is the answer.

Complexity

$O((n+m) \log n + q \log n)$ time and $O((n+m) \log n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
```

```

        st.pop_back();
        for (int v : rg[u])
            if (id[v] < 0)
                id[v] = c, st.push_back(v);
    }
    c++;
}

};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;

```

```

for (int s = 0; s < n; s++)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (auto [v, k] : l.g[u])
                if (!l.bridge[k] && id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }

    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

```

```

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
                sort(vs.begin(), vs.end());
                vs.erase(unique(vs.begin(), vs.end()), vs.end());
                int block = t.size();
                t.push_back({});
                for (int x : vs)
                    t[block].push_back(x), t[x].push_back(block);
            }
        }
    }
    return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();

```

```

    st.pop_back();
    for (int v : g[u])
        if (!reach[v])
            reach[v] = 1, st.push_back(v);
}
vector<bitset<1500>> d(n, reach);
d[0].reset();
d[0][0] = 1;
bool change = 1;
while (change) {
    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
}
return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    auto [id, t] = bridge_tree(n, e);
    TreeDistance d(t);
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << d.dist(id[--u], id[--v]) << '\n';
    }
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True

```

```

while stack:
    node, index = stack[-1]
    if index < len(graph[node]):
        target = graph[node][index]
        stack[-1] = (node, index + 1)
        if not seen[target]:
            seen[target] = True
            stack.append((target, 0))
    else:
        order.append(node)
        stack.pop()
component = [-1] * n
count = 0
for start in reversed(order):
    if component[start] != -1:
        continue
    component[start] = count
    stack = [start]
    while stack:
        node = stack.pop()
        for target in reverse[node]:
            if component[target] == -1:
                component[target] = count
                stack.append(target)
    count += 1
return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

```

```

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):
            if up[level][a] != up[level][b]:
                a, b = up[level][a], up[level][b]
        return up[0][a]

    def distance(a, b):
        ancestor = lca(a, b)
        return depth[a] + depth[b] - 2 * depth[ancestor]

    return distance

```

```

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):

```

```

        if not (reachable >> node & 1):
            continue
        value = reachable
        for parent in predecessors[node]:
            if reachable >> parent & 1:
                value &= dom[parent]
        value |= 1 << node
        if value != dom[node]:
            dom[node] = value
            changed = True

    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = data[:3]
    at = 3
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(at, at + 2 * m, 2)]
    at += 2 * m
    component, tree = bridge_tree(n, edges)
    distance = tree_lca(tree)
    print(
        "\n".join(
            str(distance(component[data[i] - 1], component[data[i + 1] - 1]))
            for i in range(at, at + 2 * q, 2)
        )
    )

if __name__ == "__main__":
    main()

```

B. Articulation Pair Damage

How to find it

Low-link separating child subtrees plus the outside remainder are precisely the components after deletion; sum cross-group pairs.

Correctness

For every child with $\text{low}[\text{child}] \geq \text{tin}[v]$, no vertex in that child subtree can escape to the rest after deleting v ; different such subtrees become different components. All vertices not in these subtrees form one remaining component. These groups partition the survivors, and a pair is disconnected exactly when its endpoints lie in distinct groups, which the incremental cross-product sum counts once.

Complexity

$O(n+m)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {

```

```

for (auto [u, v] : e)
    g[u].push_back(v), rg[v].push_back(u);
vector<char> seen(n);
for (int s = 0; s < n; s++)
    if (!seen[s]) {
        vector<pair<int, int>> st = {{s, 0}};
        seen[s] = 1;
        while (!st.empty()) {
            auto &[u, i] = st.back();
            if (i < (int)g[u].size()) {
                int v = g[u][i++];
                if (!seen[v])
                    seen[v] = 1, st.push_back({v, 0});
            } else
                order.push_back(u), st.pop_back();
        }
    }
reverse(order.begin(), order.end());
for (int s : order)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : rg[u])
                if (id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {}
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    tin[0] = low[0] = timer++;
    sub[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent_edge[u])
                continue;
            if (tin[v] >= 0)
                low[u] = min(low[u], tin[v]);
            else {
                parent_edge[v] = id;
                tin[v] = low[v] = timer++;
                sub[v] = 1;
            }
        }
    }
};

```

```

        st.push_back(v);
    }
} else {
    st.pop_back();
    int id = parent_edge[u];
    if (id < 0)
        continue;
    auto [a, b] = e[id];
    int p = a ^ b ^ u;
    sub[p] += sub[u];
    low[p] = min(low[p], low[u]);
    if (low[u] > tin[p])
        bridge[id] = 1;
    if (low[u] >= tin[p])
        parts[p].push_back(sub[u]);
}
}
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
    }
};

```

```

    }
    for (int j = 1; j < L; j++)
        for (int i = 0; i < n; i++)
            up[j][i] = up[j - 1][up[j - 1][i]];
}

int lca(int a, int b) {
    if (d[a] < d[b])
        swap(a, b);
    int z = d[a] - d[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}

int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                }
            }
        }
    }
}

```

```

        vs.push_back(e[x].second);
        if (x == id)
            break;
    }
    sort(vs.begin(), vs.end());
    vs.erase(unique(vs.begin(), vs.end()), vs.end());
    int block = t.size();
    t.push_back({});
    for (int x : vs)
        t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    LowLink l(n, e);
    for (int v = 0; v < n; v++) {
        long long ans = 0, seen = 0, sum = 0;
        for (int z : l.parts[v])
            ans += seen * z, seen += z, sum += z;
        long long rest = n - 1 - sum;
        ans += seen * rest;
    }
}

```

```

    cout << ans << " \n"[v + 1 == n];
}
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)

```

```

separating = [[] for _ in range(n)]
timer = 0

def dfs(node, parent_edge=-1):
    nonlocal timer
    tin[node] = low[node] = timer
    timer += 1
    subtree[node] = 1
    for target, edge_id in graph[node]:
        if edge_id == parent_edge:
            continue
        if tin[target] != -1:
            low[node] = min(low[node], tin[target])
        else:
            dfs(target, edge_id)
            subtree[node] += subtree[target]
            low[node] = min(low[node], low[target])
            if low[target] > tin[node]:
                bridges[edge_id] = True
            if low[target] >= tin[node]:
                separating[node].append(subtree[target])

dfs(0)
return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:

```

```

        depth[target] = depth[node] + 1
        order.append(target)
        stack.append((target, node))
for level in range(1, levels):
    previous = up[level - 1]
    up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
                elif tin[target] < tin[node]:

```

```

        edge_stack.append(edge_id)
        low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    *, separating = undirected_lowlink(n, edges)
    answer = []
    for v in range(n):
        groups = separating[v]
        rest = n - 1 - sum(groups)
        seen = 0
        value = 0
        for size in groups + [rest]:
            value += seen * size
            seen += size
        answer.append(value)
    print(*answer)

if __name__ == "__main__":
    main()

```

C. Mandatory Station Queries

How to find it

Construct vertex-biconnected blocks. The queried station is mandatory exactly when its original-vertex node lies on the block-cut-tree path.

Correctness

Every original path alternates through the biconnected blocks containing its vertices, and every block offers internal routes avoiding any other non-endpoint vertex. Consequently an original vertex is unavoidable between u and v exactly when its node separates their nodes in the block-cut tree. In a tree this is equivalent to $\text{dist}(u,v)=\text{dist}(u,c)+\text{dist}(c,v)$.

Complexity

$O((n+m) \log n + q \log n)$ time and $O((n+m) \log n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)
                            id[v] = c, st.push_back(v);
                }
                c++;
            }
    }
};
struct LowLink {
```

```

int n, timer = 0;
vector<pair<int, int>> e;
vector<vector<pair<int, int>>> g;
vector<int> tin, low, sub, parent_edge, it;
vector<char> bridge;
vector<vector<int>> parts;
LowLink(int n, vector<pair<int, int>> e)
    : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
      bridge(e.size()), parts(n) {}
for (int i = 0; i < (int)e.size(); i++) {
    auto [u, v] = e[i];
    g[u].push_back({v, i});
    g[v].push_back({u, i});
}
tin[0] = low[0] = timer++;
sub[0] = 1;
vector<int> st = {0};
while (!st.empty()) {
    int u = st.back();
    if (it[u] < (int)g[u].size()) {
        auto [v, id] = g[u][it[u]++];
        if (id == parent_edge[u])
            continue;
        if (tin[v] >= 0)
            low[u] = min(low[u], tin[v]);
        else {
            parent_edge[v] = id;
            tin[v] = low[v] = timer++;
            sub[v] = 1;
            st.push_back(v);
        }
    } else {
        st.pop_back();
        int id = parent_edge[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        sub[p] += sub[u];
        low[p] = min(low[p], low[u]);
        if (low[u] > tin[p])
            bridge[id] = 1;
        if (low[u] >= tin[p])
            parts[p].push_back(sub[u]);
    }
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    return {id, l.parts};
}

```

```

    }
    c++;
}
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }
    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;

```

```

tin[0] = low[0] = timer++;
vector<int> dfs = {0};
while (!dfs.empty()) {
    int u = dfs.back();
    if (it[u] < (int)g[u].size()) {
        auto [v, id] = g[u][it[u]++];
        if (id == parent[u])
            continue;
        if (tin[v] < 0) {
            parent[v] = id;
            edges.push_back(id);
            tin[v] = low[v] = timer++;
            dfs.push_back(v);
        } else if (tin[v] < tin[u])
            edges.push_back(id), low[u] = min(low[u], tin[v]);
    } else {
        dfs.pop_back();
        int id = parent[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        low[p] = min(low[p], low[u]);
        if (low[u] >= tin[p]) {
            vector<int> vs;
            while (1) {
                int x = edges.back();
                edges.pop_back();
                vs.push_back(e[x].first);
                vs.push_back(e[x].second);
                if (x == id)
                    break;
            }
            sort(vs.begin(), vs.end());
            vs.erase(unique(vs.begin(), vs.end()), vs.end());
            int block = t.size();
            t.push_back({});
            for (int x : vs)
                t[block].push_back(x), t[x].push_back(block);
        }
    }
}
return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {

```

```

    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    TreeDistance d(block_cut(n, e));
    while (q--) {
        int u, v, c;
        cin >> u >> v >> c;
        --u;
        --v;
        --c;
        cout << (d.dist(u, v) == d.dist(u, c) + d.dist(c, v) ? "YES\n" : "NO\n");
    }
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    return order

```

```

        else:
            order.append(node)
            stack.pop()
component = [-1] * n
count = 0
for start in reversed(order):
    if component[start] != -1:
        continue
    component[start] = count
    stack = [start]
    while stack:
        node = stack.pop()
        for target in reverse[node]:
            if component[target] == -1:
                component[target] = count
                stack.append(target)
    count += 1
return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count

```

```

stack = [start]
while stack:
    node = stack.pop()
    for target, edge_id in graph[node]:
        if not bridges[edge_id] and component[target] == -1:
            component[target] = count
            stack.append(target)
    count += 1
tree = [[] for _ in range(count)]
for edge_id, (u, v) in enumerate(edges):
    if bridges[edge_id]:
        a, b = component[u], component[v]
        tree[a].append(b)
        tree[b].append(a)
return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):
            if up[level][a] != up[level][b]:
                a, b = up[level][a], up[level][b]
        return up[0][a]

    def distance(a, b):
        ancestor = lca(a, b)
        return depth[a] + depth[b] - 2 * depth[ancestor]

    return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n

```

```

low = [0] * n
edge_stack = []
tree = [[] for _ in range(n)]
timer = 0

def dfs(node, parent_edge=-1):
    nonlocal timer
    tin[node] = low[node] = timer
    timer += 1
    for target, edge_id in graph[node]:
        if edge_id == parent_edge:
            continue
        if tin[target] == -1:
            edge_stack.append(edge_id)
            dfs(target, edge_id)
            low[node] = min(low[node], low[target])
            if low[target] >= tin[node]:
                vertices = set()
                while True:
                    taken = edge_stack.pop()
                    vertices.update(edges[taken])
                    if taken == edge_id:
                        break
                block = len(tree)
                tree.append([])
                for vertex in vertices:
                    tree[block].append(vertex)
                    tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

dfs(0)
return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node

```

```

        if value != dom[node]:
            dom[node] = value
            changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = data[:3]
    at = 3
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(at, at + 2 * m, 2)]
    at += 2 * m
    distance = tree_lca(block_cut_tree(n, edges))
    out = []
    for i in range(at, at + 3 * q, 3):
        u, v, c = data[i] - 1, data[i + 1] - 1, data[i + 2] - 1
        out.append(
            "YES"
            if distance(u, v) == distance(u, c) + distance(c, v)
            else "NO"
        )
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

D. Two-Edge Connectivity Completion

How to find it

Every leaf of the bridge tree needs a new route; one added edge can repair two leaves, and pairing leaves attains the lower bound.

Correctness

Each leaf component of the bridge tree has only one old edge to the remainder, so any bridgeless augmentation must give it at least one new incident route. One added edge can serve at most two leaves, giving $\text{ceil}(\text{leaves}/2)$ as a lower bound. Pairing leaves across the tree creates cycles covering their incident bridge paths and attains this bound.

Complexity

$O(n+m)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};

```

```

        seen[s] = 1;
        while (!st.empty()) {
            auto &[u, i] = st.back();
            if (i < (int)g[u].size()) {
                int v = g[u][i++];
                if (!seen[v])
                    seen[v] = 1, st.push_back({v, 0});
            } else
                order.push_back(u), st.pop_back();
        }
    }
    reverse(order.begin(), order.end());
    for (int s : order)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (int v : rg[u])
                    if (id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    };
    struct LowLink {
        int n, timer = 0;
        vector<pair<int, int>> e;
        vector<vector<pair<int, int>>> g;
        vector<int> tin, low, sub, parent_edge, it;
        vector<char> bridge;
        vector<vector<int>> parts;
        LowLink(int n, vector<pair<int, int>> e)
            : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
              bridge(e.size()), parts(n) {
            for (int i = 0; i < (int)e.size(); i++) {
                auto [u, v] = e[i];
                g[u].push_back({v, i});
                g[v].push_back({u, i});
            }
            tin[0] = low[0] = timer++;
            sub[0] = 1;
            vector<int> st = {0};
            while (!st.empty()) {
                int u = st.back();
                if (it[u] < (int)g[u].size()) {
                    auto [v, id] = g[u][it[u]++];
                    if (id == parent_edge[u])
                        continue;
                    if (tin[v] >= 0)
                        low[u] = min(low[u], tin[v]);
                    else {
                        parent_edge[v] = id;
                        tin[v] = low[v] = timer++;
                        sub[v] = 1;
                        st.push_back(v);
                    }
                } else {
                    st.pop_back();
                    int id = parent_edge[u];
                    if (id < 0)

```

```

        continue;
    auto [a, b] = e[id];
    int p = a ^ b ^ u;
    sub[p] += sub[u];
    low[p] = min(low[p], low[u]);
    if (low[u] > tin[p])
        bridge[id] = 1;
    if (low[u] >= tin[p])
        parts[p].push_back(sub[u]);
    }
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }
    int lca(int a, int b) {

```

```

    if (d[a] < d[b])
        swap(a, b);
    int z = d[a] - d[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}
int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
                sort(vs.begin(), vs.end());
                vs.erase(unique(vs.begin(), vs.end()), vs.end());
            }
        }
    }
}

```

```

        int block = t.size();
        t.push_back({});
        for (int x : vs)
            t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    auto [id, t] = bridge_tree(n, e);
    int leaves = 0;
    for (auto &v : t)
        leaves += v.size() == 1;
    cout << (leaves + 1) / 2 << '\n';
}

```

Python

```
import sys
```

```

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:

```

```

        continue
    if tin[target] != -1:
        low[node] = min(low[node], tin[target])
    else:
        dfs(target, edge_id)
        subtree[node] += subtree[target]
        low[node] = min(low[node], low[target])
        if low[target] > tin[node]:
            bridges[edge_id] = True
        if low[target] >= tin[node]:
            separating[node].append(subtree[target])

dfs(0)
return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a

```

```

    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]

```

```

for u, v in edges:
    graph[u].append(v)
    predecessors[v].append(u)
reachable = 1
stack = [0]
while stack:
    node = stack.pop()
    for target in graph[node]:
        if not (reachable >> target & 1):
            reachable |= 1 << target
            stack.append(target)
dom = [reachable] * n
dom[0] = 1
changed = True
while changed:
    changed = False
    for node in range(1, n):
        if not (reachable >> node & 1):
            continue
        value = reachable
        for parent in predecessors[node]:
            if reachable >> parent & 1:
                value &= dom[parent]
        value |= 1 << node
        if value != dom[node]:
            dom[node] = value
            changed = True
return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    _, tree = bridge_tree(n, edges)
    leaves = sum(len(row) == 1 for row in tree)
    print((leaves + 1) // 2)

if __name__ == "__main__":
    main()

```

Upsolving and stress testing

For A, delete each physical edge ID and compare component counts; parallel edges are the decisive case. For B, delete every candidate vertex and count separated unordered pairs directly. For C, enumerate simple paths and compare their intersection with block-cut path membership. For D, enumerate added edges on tiny component forests and retest bridge-freedom.

Every proof should begin with the exact allowed escape route encoded by `low`. From that one invariant, the strict bridge inequality, non-strict articulation boundary, and edge-stack block extraction follow with different roles. Copying one inequality into another problem is a common near-correct failure.